

INCIDENT RESPONSE PLAYBOOK

Network Forensics with Wireshark

Version 1.0

Classification: Training Material

Audience: Security Analysts, IR Teams, Forensic Investigators

A comprehensive guide to network forensic analysis
using Wireshark and tshark for incident response.

INCIDENT RESPONSE PLAYBOOK: NETWORK FORENSICS WITH WIRESHARK

1.0

Training Material

Wireshark + tshark

Security Analysts, IR Teams, Forensic Investigators

TABLE OF CONTENTS

- 1. The IR Forensic Mindset
- 2. Phase 0 — Preparation: Before the Incident
- 3. Phase 1 — Identification: Detecting the Incident
- 4. Phase 2 — Containment: Capturing Network Evidence
- 5. Phase 3 — Triage: First Pass Analysis
- 6. Phase 4 — Deep Dive: Full Forensic Analysis
- 7. Phase 5 — Evidence Extraction & IOCs
- 8. Phase 6 — Timeline Reconstruction
- 9. Phase 7 — Reporting & Documentation
- 10. The Wireshark Filter Cheat Sheet
- 11. tshark Quick Reference
- 12. Attack Pattern Recognition Guide
- 13. Legal & Chain of Custody
- 14. Appendix A — Dactical Exercises
- 15. Appendix B — Tool Arsenal

1. THE IR FORENSIC MINDSET

The Core Principles

Principle	Meaning
Preserve Before You Analyze	Never work on the original pcap. Always work on a copy.
Assume Everything is Evidence	Traffic that looks innocent may be the attack.
Follow the Data, Not the Assumptions	Let the packets tell you what happened.
Correlate or Die	Single packets tell stories. Patterns tell truths.
Document Everything	If it's not written down, it didn't happen.
Speed Matters, Accuracy Matters More	

Be fast but never skip steps.

The Forensic Question Stack

Every investigation answers these questions in order:

1. WHAT happened?

→ Events reconstructed
2. WHEN did it happen?

→ Timeline
3. WHO did it?

→ Attribution (IPs, MACs, infrastructure)
4. WHERE did it happen?

→ Network topology, affected hosts
5. HOW did it happen?

→ Attack vector, tools, techniques
6. WHY did it happen?

→ Motive (data theft, persistence, reconnaissance)
7. WHAT was the impact?

→ Data exfiltrated, systems compromised, lateral movement

The Three-Truth Rule

In network forensics, you seek three independent sources of truth:

- what the packets say
- what the endpoint logs/artifacts say
- what external feeds say about IOCs

When all three align, you have high confidence. When they diverge, you have a deeper problem.

2. PHASE 0 — PREPARATION: BEFORE THE INCIDENT

2.1 Lab Environment Setup

Tool	Purpose	Install
Wireshark	GUI analysis	sudo pacman -S wireshark-qt
tshark	CLI analysis, scripting, automation	sudo pacman -S wireshark-cli
tcpdump	Lightweight capture	sudo pacman -S tcpdump
binwalk	Firmware/file carving	sudo pacman -S binwalk
foremost	File carving from pcaps	sudo pacman -S foremost
NetworkMiner	Auto artifact extraction	AUR or download
zeek (Bro)	Protocol analysis at scale	sudo pacman -S zeek
suricata	IDS/IPS signature detection	sudo pacman -S suricata

volatility

Memory forensics (correlate)

pip install volatility3 (in venv)

2.2 Capture Strategy

on core switch captures all LAN traffic

inline, fail-safe, captures full-duplex

tcpdump on suspect hosts

captures all inbound/outbound

for cloud environments

```
# Full capture (small networks):
tcpdump -i eth0 -w incident-$(date +%Y%m%d-%H%M).pcap

# No broadcast/multicast (saves space):
tcpdump -i eth0 -w incident.pcap not broadcast and not multicast

# Specific suspect host:
tcpdump -i eth0 -w suspect.pcap host 192.168.1.42

# Specific subnet:
tcpdump -i eth0 -w subnet.pcap net 192.168.1.0/24

# Rotate files every 100MB or 1 hour:
tcpdump -i eth0 -w incident-%Y%m%d-%H%M.pcap -C 100 -W 50 -G 3600
```

2.3 Pre-Incident Baseline

Document before an incident:

- Normal DNS servers used
- Normal outbound ports
- Normal external partners/CDNs
- Normal bandwidth patterns
- Normal protocol breakdown
- DHCP ranges
- Critical asset IPs and MACs

2.4 Chain of Custody Kit

Prepare these forms:

- description, source, datetime, hash
- who handled evidence, when, why
- what was done, what was found

```
sha256sum incident.pcap > incident.pcap.sha256
md5sum incident.pcap > incident.pcap.md5
```

3. PHASE 1 — IDENTIFICATION: DETECTING THE INCIDENT

3.1 Common Detection Sources

Source	What It Tells You	Priority
IDS/IPS alert (Suricata/Snort)	Signature match, known attack pattern	HIGH
SIEM correlation	Multiple low-level events combining into a threat	HIGH
Endpoint EDR	Process anomaly, file drop, registry change	HIGH
Firewall logs	Unusual port access, blocked traffic, geo anomalies	MED
User report	"My computer is acting weird"	MED
DNS logs	DGA domains, tunneling, excessive queries	MED
NetFlow data	Volume anomalies, beaconing patterns	HIGH
Threat intel feed	Known C2 IPs/domains hitting your network	HIGH

3.2 Triage Questions — What to Ask When an Alert Fires

1. What alert fired? (SID, rule name, description)
2. Which host(s) are involved?
3. Is this a known critical asset?
4. What time did the alert fire?
5. Is the traffic still ongoing?

6. Has this been seen before? (historical baseline)
7. What is the source IP? Internal or external?
8. What is the destination port?
9. Is geolocation relevant? (unexpected country)
10. What is the business impact? (asset criticality)

3.3 Decision Tree — Is This an Incident?

```
Alert fires
├─ Expected traffic? (check baseline)
│   ├── YES → Document and close (false positive)
│   └─ NO → Continue investigation
├─ Known threat signature?
│   ├── YES → Escalate to IR, begin capture
│   └─ NO → Behavioral analysis needed → Continue
├─ Internal-to-internal traffic on unusual ports?
│   ├── YES → Possible lateral movement → Escalate
│   └─ NO → Continue
├─ Outbound traffic to unknown external IP?
│   ├── YES → Possible C2 or exfiltration → Escalate
│   └─ NO → Continue
└─ Excessive traffic volume or beaconing pattern?
    ├── YES → Possible C2 beacon or data exfil → Escalate
    └─ NO → Document and monitor
```

4. PHASE 2 — CONTAINMENT: CAPTURING NETWORK EVIDENCE

4.1 Live Capture During an Active Incident

1. Start capture immediately on the affected segment
2. Do NOT reboot or disconnect the host (yet) — capture first
3. Capture at all key points: gateway, suspect host, core switch
4. Save to non-volatile storage (not /tmp)

```
# Immediate full capture on the segment:
sudo tcpdump -i eth0 -w /cases/incident-001/full-$(date +%Y%m%d-%H%M%S).pcap -W 100 -C 100

# Capture specific suspect host only:
sudo tcpdump -i eth0 -w /cases/incident-001/suspect.pcap host 10.0.0.42

# Capture with no hostname resolution (faster):
sudo tcpdump -i eth0 -nn -w /cases/incident-001/full.pcap -W 100 -C 100
```

4.2 What NOT to Do

Don't	Do
Reboot the suspect host	Capture traffic first, then isolate
Disconnect from network immediately	Capture live traffic before disconnecting
Analyze the original pcap	Copy it and work on the copy
Delete "irrelevant" packets	Keep everything — it may matter later
Store evidence on the suspect machine	Use external verified storage
Use unencrypted storage for sensitive captures	Encrypt evidence store (LUKS, GPG)

4.3 Evidence Preservation

```
# Step 1: Copy original to working copy
cp incident.pcap incident-working.pcap

# Step 2: Hash both files
sha256sum incident.pcap > incident.pcap.sha256
sha256sum incident-working.pcap > incident-working.pcap.sha256

# Step 3: Record chain of custody
# - Who captured it (name, role)
# - When (date, time, timezone)
# - Where (which sensor/interface)
# - What (description of capture)
# - Hash values

# Step 4: Store original in evidence safe
# - Offline storage (USB, external HDD)
# - Read-only mount
# - Physical security
```

5. PHASE 3 — TRIAGE: FIRST PASS ANALYSIS

5.1 The 5-Minute Triage (First Look in Wireshark)

Within 5 minutes, determine if this is a real incident, and if so, what type.

Step 1: File Overview

- Open pcap in Wireshark
- in bottom — check packet count, duration, file size
- File File Properties — confirms file integrity, hash, comments

Step 2: Protocol Hierarchy

- Statistics Protocol Hierarchy
- Look for:
 - (Telnet, IRC, SMB in a web environment)
 - (DNS over TCP instead of UDP, HTTP on non-standard ports)
 - — high TLS may hide C2

Step 3: Conversations

- Statistics Conversations

- tab:
- Who is talking to who?
- Any unusual port combinations?
- One host connecting to many ports on another? (port scan)
- Many hosts connecting to one external IP? (C2)
- tab:
- External IPs — are any unexpected?
- Internal IPs — are all within expected ranges?

Step 4: Endpoints

- Statistics Endpoints
- Sort by bytes — who's sending/receiving the most data?
- Look for:
 - is the expected top talker, or is a compromised host?
 - possible exfiltration

Step 5: I/O Graph

- Statistics I/O Graphs
- Look for:
 - burst of activity at a specific time
 - beaconing (regular intervals)
 - bulk data transfer
- What protocols are present
- Who the key players are
- Whether traffic patterns look malicious
- What sections need deep-dive

5.2 The Triage Conclusion

Finding	Likely Classification	Next Step
Telnet/SSH from external IP	Remote access attack	Follow streams, extract commands
DNS with high entropy queries	DNS tunneling or DGA	Analyze DNS queries, extract data
HTTP with encoded payloads	C2 or exfiltration	Decode payloads, extract IOCs
Repeated SYN to many ports	Port scan / enumeration	Map scan target, assess exposure
Large outbound HTTP/FTP	Data exfiltration	Export objects, identify data
Beaconing pattern (regular gaps)	C2 callback	Calculate beacon interval, identify C2

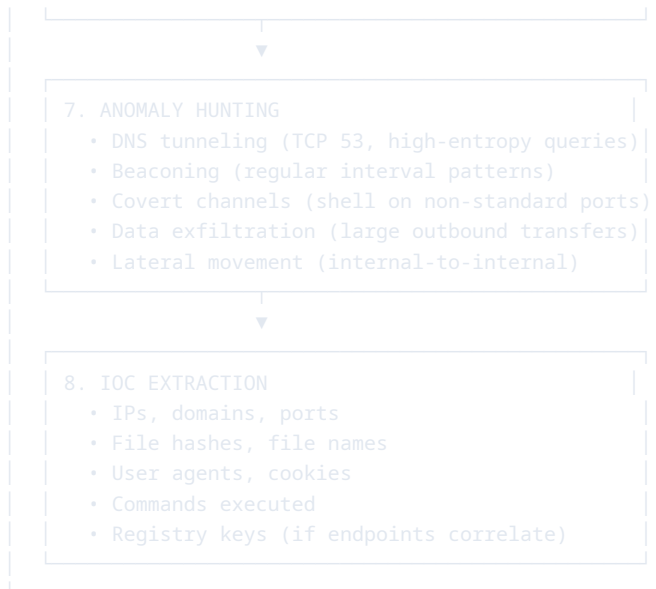
ARP storm	ARP poisoning / MITM	Check for duplicate MACs, ARP table
Batch sessions on unusual ports	Backdoor / reverse shell	Follow streams, decode commands

6. PHASE 4 — DEEP DIVE: FULL FORENSIC ANALYSIS

6.1 The Wireshark Deep Dive Workflow

This is the core methodology — the step-by-step process you follow for every pcap.





6.2 Follow TCP Stream — The #1 Skill

- It reconstructs the full conversation between two hosts
 - It shows you the actual commands, responses, and data
 - It decodes the raw bytes into readable text
 - It color-codes client vs server traffic
 - It reveals credentials, commands, exfiltrated data
1. Filter to the conversation of interest (tcp.port == X)
 2. Right-click any packet in that conversation
 3. Follow TCP Stream
 4. A new window opens with the full conversation
 - = Server Client (responses, banners, output)
 - = Client Server (commands, requests, data uploads)
 - = Direction unclear or mixed
- In the Follow Stream window, click Save as
 - Save as .txt for evidence preservation
 - Name it: stream----.txt

After following a stream, Wireshark auto-applies a filter like:

```
tcp.stream eq 5
```

You can manually type this to jump directly to a specific stream.

6.3 Protocol-Specific Deep Dives

6.3.1 DNS Analysis

- UDP port 53
- Query Response pattern
- Hostnames are readable
- Small payloads (< 512 bytes typically)

Pattern	Filter	What it Means
DNS over TCP	tcp.port == 53 && dns	Unusual — may be tunneling
High-entropy query names	dns.qry.name — look at values	DNS tunneling — data exfiltration via query names
Many queries to same domain	dns.qry.name contains "evil.com"	C2 via DNS
TXT record abuse	dns.txt	Data exfiltration via TXT records
Long query labels	Manual inspection	Each label > 63 chars is suspicious
Excessive NXDOMAIN responses	dns.flags.rcode == 3	DGA (domain generation algorithm)
DNS requests to non-standard servers	Check destination IPs	Bypassing internal DNS — direct to attacker

```
Filter: udp.port == 53

Look for:
1. Query names that are random-looking (base32/hex encoded)
   Example: JFGHJKDS732NDJA.evil.com — likely encoded data
2. Frequent queries in short time (burst pattern)
3. TXT responses with long strings
4. Queries to domains you don't recognize
5. One host making 100s of DNS queries to the same domain
```

```
# All DNS queries and responses:
tshark -r incident.pcap -Y "dns" -T fields \
  -e frame.number -e frame.time -e ip.src -e ip.dst \
  -e dns.qry.name -e dns.a -e dns.flags.response

# Unique domains queried:
tshark -r incident.pcap -Y "dns.flags.response == 0" \
  -T fields -e dns.qry.name | sort -u

# DNS queries to specific domain:
tshark -r incident.pcap -Y 'dns.qry.name contains "evil"' \
  -T fields -e frame.number -e ip.src -e dns.qry.name
```

6.3.2 HTTP Analysis

```
# All HTTP requests:
tshark -r incident.pcap -Y "http.request" -T fields \
  -e frame.number -e ip.src -e ip.dst \
  -e http.request.method -e http.host -e http.request.uri

# All HTTP responses (status codes):
tshark -r incident.pcap -Y "http.response" -T fields \
  -e frame.number -e ip.src -e http.response.code -e http.content_type

# POST requests (data upload):
tshark -r incident.pcap -Y "http.request.method == POST" -T fields \
  -e frame.number -e ip.src -e http.host -e http.request.uri
```

- Filter: http.request — all web requests
- Filter: http.response.code == 200 — successful responses
- Filter: http.request.method == POST — data being sent
- File Export Objects HTTP — extract transferred files
- malware often has unique UAs
- Encoded URLs (%2f, base64 in path)
- POST to unknown external domains
- Large HTTP responses (file downloads)
- Unusual Content-Type (e.g., application/octet-stream = binary download)

6.3.3 TLS/SSL Analysis

- SNI (Server Name Indication) — the domain being requested
- Certificate details — issuer, subject, validity
- Cipher suite negotiation
- TLS version
- Traffic volume and timing (metadata)
- The actual content (encrypted)

```
# All TLS handshakes:
tls.handshake.type == 1

# Server Name (SNI):
tls.handshake.extensions_server_name

# Certificate:
tls.handshake.certificate

# TLS version:
tls.record.version == 0x0301  (TLS 1.0)
tls.record.version == 0x0303  (TLS 1.2)

# Specific SNI:
tls.handshake.extensions_server_name contains "evil"

# Extract all SNIs:
tshark -r incident.pcap -Y "tls.handshake.type == 1" -T fields \
  -e ip.src -e ip.dst -e tls.handshake.extensions_server_name

# Extract certificate subjects:
tshark -r incident.pcap -Y "tls.handshake.certificate" -T fields \
  -e x509ce.distinguishedName
```

6.3.4 SMB Analysis (Lateral Movement)

```
# SMB2 commands:
smb2.cmd == 5  (CREATE – file access)
smb2.cmd == 6  (CLOSE)
smb2.cmd == 8  (READ)
smb2.cmd == 9  (WRITE)

# File copy via SMB:
smb2.filename
smb2.file_name

# Administrative shares:
smb2.filename contains "C$"
smb2.filename contains "ADMIN$"
```

6.3.5 Telnet Analysis

```
# All Telnet traffic:
telnet

# Telnet with data:
telnet.data

# Follow stream for full session:
Right-click → Follow → TCP Stream
```

- Login banners (OS version, hostname)
- Username/password prompts
- Commands entered
- Data returned
- Any privilege escalation attempts

6.4 Anomaly Hunting Techniques

6.4.1 Beaconsing Detection

= regular, periodic connections to a C2 server.

1. Identify suspect host external IP
2. Filter: `ip.addr == && ip.addr ==`
3. Look at the time column — are connections at regular intervals?
4. Statistics Flow Graph — visualize the timing
5. Statistics TCP Flow Graph — connection patterns

```
# Timestamps of all connections to a specific IP:
tshark -r incident.pcap -Y "ip.dst == 10.1.2.3 && tcp.flags.syn == 1" \
-T fields -e frame.time | sort

# Calculate intervals in Python:
# (feed the timestamps above into a script that computes delta between each)
```

- Regular intervals (e.g., every 30s, 60s, 300s)
- Small, similar-sized packets
- Same destination IP and port
- Often over HTTP or DNS

6.4.2 Data Exfiltration Detection

- Large outbound transfer to external IP
- Non-standard ports for bulk transfer
- Encrypted traffic to unknown endpoints
- DNS queries with large payloads
- HTTP POST with large body

```
# Large HTTP POST bodies:
http.request.method == POST && http.content_length > 10000

# Long DNS queries (tunneling):
udp.length > 200

# Specific large outbound transfer:
ip.src == 192.168.1.42 && tcp.len > 1000
```

- Set Y axis to bytes
- Look for large outbound spikes
- Filter to specific host to isolate

6.4.3 Lateral Movement Detection

- Internal host connecting to other internal hosts on admin ports

- SMB, RDP, WMI, WinRM traffic between workstations
- PsExec, WMI, or DCOM activity
- New service Creation (SCM)
- Authentication bursts (pass-the-hash)

```
# SMB between internal hosts:
ip.src == 192.168.1.0/24 && ip.dst == 192.168.1.0/24 && smb

# RDP traffic:
tcp.port == 3389

# WinRM:
tcp.port == 5985 || tcp.port == 5986

# PsExec (uses SMB named pipes):
smb2.filename contains "psexesvc"
```

7. PHASE 5 — EVIDENCE EXTRACTION & IOCs

7.1 What to Extract

Evidence Type	Method	Tool
Transferred files	File Export Objects	Wireshark
Credentials (cleartext)	Follow Stream, search	Wireshark
Commands executed	Follow Stream on shell sessions	Wireshark
DNS domains	dns.qry.name field extraction	tshark
IP addresses	Endpoint/conversation export	Both
File hashes	Export then hash externally	Export + sha256sum
User-Agent strings	http.user_agent field	tshark
Certificates	TLS certificate extraction	tshark
Cookies	http.cookie field	tshark

URLs		
	http.request.full_uri	
		tshark

7.2 tshark IOC Extraction Cheatsheet

```
# ===== IPs =====
# All unique IPs:
tshark -r incident.pcap -T fields -e ip.src -e ip.dst | \
  tr '\t' '\n' | sort -u

# ===== DOMAINS =====
# All DNS queries:
tshark -r incident.pcap -Y "dns.flags.response == 0" \
  -T fields -e dns.qry.name | sort -u

# ===== URLs =====
# All HTTP URLs:
tshark -r incident.pcap -Y "http.request" -T fields \
  -e http.host -e http.request.uri | sort -u

# ===== USER AGENTS =====
tshark -r incident.pcap -Y "http.user_agent" -T fields \
  -e http.user_agent | sort -u

# ===== CREDENTIALS =====
# HTTP Basic Auth:
tshark -r incident.pcap -Y "http.authorization" -T fields \
  -e ip.src -e http.authorization

# FTP credentials:
tshark -r incident.pcap -Y "ftp.request.command == USER || ftp.request.command == PASS" \
  -T fields -e ip.src -e ftp.request.arg

# ===== FILES =====
# HTTP objects:
tshark -r incident.pcap --export-objects http,/tmp/http-objects/

# SMB objects:
tshark -r incident.pcap --export-objects smb,/tmp/smb-objects/

# ===== CERTIFICATES =====
tshark -r incident.pcap -Y "tls.handshake.certificate" -T fields \
  -e ip.src -e x509ce.distinguishedName

# ===== FULL CONVERSATIONS =====
# Follow stream via tshark:
tshark -r incident.pcap -q -z follow,tcp,ascii,0
# (0 = stream index, change for each stream)
```

7.3 IOC Output Format

Type	Value	Context	Source Frame
IP	192.168.1.3	Attacker — internal lateral source	All

IP	83.170.75.178	External HTTP — web1.goals365.com	F74
Domain	web1.goals365.com	Resolved to 83.170.75.178	F74
Port	TCP 53	Covert shell channel (non-DNS)	F14-38
Port	TCP 23	Telnet backdoor	F76-108
File	installer-debug.txt	On victim C:\ — 11,531 bytes	F89
Command	dir	Executed by attacker	F25, F87, F119
Command	ls -la	Failed — attacker Linux-adapted	F97
MAC	00:80:48:24:33:32	Victim NIC — Premier Imaging	F13
OS	Windows XP 5.1.2600	Victim OS from Telnet banner	F21, F83
Hash	SHA256 of pcap	Evidence integrity	File properties

8. PHASE 6 — TIMELINE RECONSTRUCTION

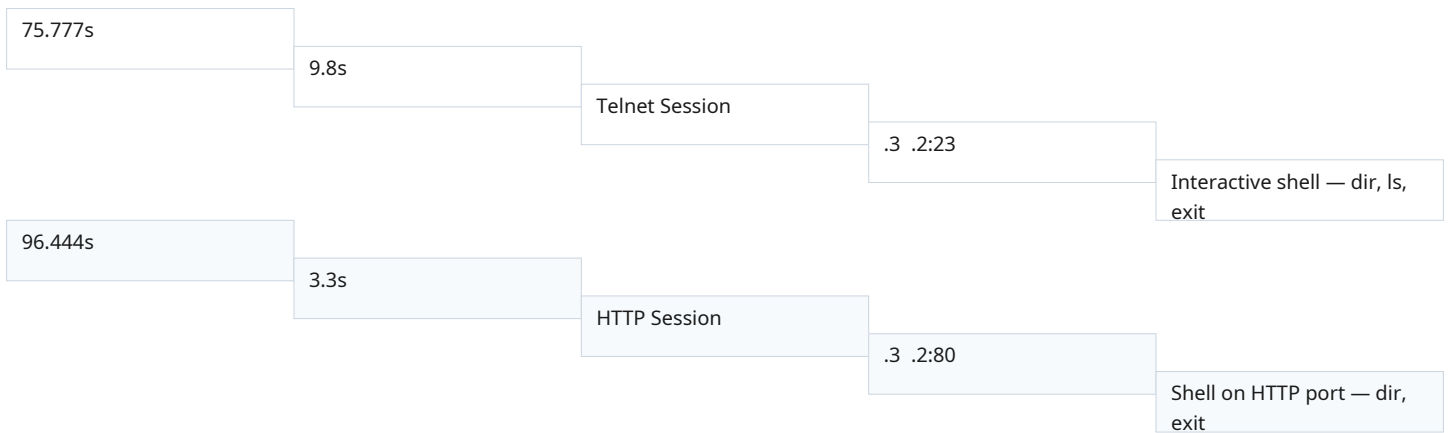
8.1 How to Build a Timeline

```
tshark -r incident.pcap -T fields \
  -e frame.number -e frame.time -e ip.src -e ip.dst \
  -e _ws.col.Protocol -e _ws.col.Info > timeline_raw.tsv
```

Group individual packets into logical events:

- All packets in a TCP conversation = one event
- DNS query+response = one event
- ARP exchange = one event
- SSH session = one event

Time	Duration	Event	Source	Dest	Details
0.000s	—	ARP Request	.3 .1		Who has 192.168.1.1?
0.017s	—	DNS PTR Query	.3 .1		Reverse lookup for 192.168.1.1
0.020s	—	DNS PTR Response	.1 .3		SpeedTouch.lan
2.629s	—	DNS A Query	.3 .1		www.www.com.lan
25.493s	5.3s	TCP Session	.3 .2:53		Remote shell on DNS port
48.475s	6.1s	Port Scan	.3 .2:21		6 SYN 6 RST (FTP closed)



Map events to the attack lifecycle:

Recon → Initial Access → Execution → Persistence → Lateral Movement → C2 → Exfiltration

Not every phase will be present in every pcap — but map what you see.

8.2 Visual Timeline with Wireshark

1. Statistics Flow Graph
2. This shows a visual timeline of all conversations
3. You can see the sequence of events at a glance
4. Filter to specific hosts/ports first for clarity

1. Statistics I/O Graphs
2. Shows traffic volume over time
3. Spikes = bursts of activity (commands, file transfers)
4. Regular patterns = beaconing

9. PHASE 7 — REPORTING & DOCUMENTATION

9.1 Report Structure

1. EXECUTIVE SUMMARY
 - What happened (1 paragraph, non-technical)
 - Impact assessment (1 paragraph)
 - Confidence level (High/Medium/Low)
2. INCIDENT DETAILS
 - Detection source (how was it detected)
 - First observed (timestamp)
 - Last observed (timestamp)
 - Affected assets (IPs, hostnames, users)
3. TECHNICAL ANALYSIS
 - Network overview (topology, protocols, duration)
 - Timeline of events (the table from Phase 6)
 - Detailed analysis of each session/conversation
 - Payloads decoded (commands, credentials, files)
 - Anomalies identified

4. INDICATORS OF COMPROMISE
 - Table of all IOCs (IPs, domains, ports, files, hashes)
 - Confidence level per IOC
 - MISP/STIX format if required
5. ATTACK CHAIN MAPPING
 - Map events to MITRE ATT&CK tactics
 - Identify techniques used
6. IMPACT ASSESSMENT
 - Data potentially exfiltrated
 - Systems compromised
 - Business operations affected
7. RECOMMENDATIONS
 - Immediate containment actions
 - Short-term remediation
 - Long-term improvements
8. EVIDENCE APPENDIX
 - Pcap file name, hash, size
 - Chain of custody form
 - Screenshots of key findings
 - Full Follow Stream outputs

9.2 MITRE ATT&CK Mapping

Map every finding to ATT&CK for structured reporting:

ATTACK Finding	Tactic	Technique
ARP + DNS queries	Reconnaissance	Active Scanning (T1595)
Port 21 connection attempts	Reconnaissance	Scanning IP Blocks (T1595.001)
Remote shell on port 53	Command & Control	Non-Standard Port (T1571)
Remote shell on port 23	Execution	Remote Services (T1021)
Remote shell on port 80	Command & Control	Web Protocols (T1071.001)
dir command execution	Discovery	File and Directory Discovery (T1083)
Windows XP banner	Discovery	System Information Discovery (T1082)
External SSL connection	Command & Control	Encrypted Channel (T1573)

HTTP GET to external IP

Command & Control

Application Layer Protocol (T1071)

10. THE WIRESHARK FILTER CHEAT SHEET

10.1 Essential Filters

```
# === PROTOCOL FILTERS ===
arp                # ARP traffic
dns                # DNS protocol
http               # HTTP protocol
https || tls       # TLS/HTTPS
telnet             # Telnet
ftp                # FTP
smb || smb2        # SMB file sharing
rdp                # Remote Desktop
ssh                # SSH
icmp               # Ping
tcp.port == 22     # SSH port
tcp.port == 23     # Telnet port
tcp.port == 53     # DNS port (TCP)
udp.port == 53     # DNS port (UDP)
tcp.port == 80     # HTTP port
tcp.port == 443    # HTTPS port
tcp.port == 445    # SMB port
tcp.port == 3389   # RDP port

# === IP FILTERS ===
ip.addr == 192.168.1.3      # Traffic to OR from this IP
ip.src == 192.168.1.3      # Traffic FROM this IP
ip.dst == 192.168.1.3      # Traffic TO this IP
ip.addr == 192.168.1.0/24  # Traffic in this subnet
!(ip.addr == 192.168.1.0/24) # Traffic OUTSIDE this subnet (external)
ip.src == 192.168.1.3 && ip.dst == 192.168.1.2 # One direction only

# === TCP FLAG FILTERS ===
tcp.flags.syn == 1 && tcp.flags.ack == 0 # SYN-only (connection attempt)
tcp.flags.syn == 1 && tcp.flags.ack == 1 # SYN-ACK (open port response)
tcp.flags.reset == 1                    # RST (connection reset/rejected)
tcp.flags.push == 1                     # PSH (data being sent)
tcp.flags.fin == 1                      # FIN (graceful close)
tcp.flags == 0x000                      # NULL scan (no flags)
tcp.flags == 0x029                      # XMAS scan (FIN+URG+PSH)

# === PORT SCAN DETECTION ===
# SYN scan (one SYN, no ACK, to many ports):
ip.src == 10.0.0.5 && tcp.flags.syn == 1 && tcp.flags.ack == 0

# See all rejected connections:
tcp.flags.reset == 1 && tcp.flags.ack == 1

# === DNS ANOMALY FILTERS ===
# DNS over TCP (unusual):
tcp.port == 53 && dns

# DNS responses with error codes:
dns.flags.rcode == 3      # NXDOMAIN (domain doesn't exist)
dns.flags.rcode == 2      # SERVFAIL

# Long DNS queries (potential tunneling):
```

```

udp.length > 100

# DNS TXT records (potential data exfiltration):
dns.txt

# === HTTP FILTERS ===
http.request           # All HTTP requests
http.response          # All HTTP responses
http.request.method == "GET"      # GET requests
http.request.method == "POST"    # POST requests (data upload)
http.response.code == 200        # Successful responses
http.response.code == 302        # Redirects
http.response.code >= 400        # Errors (403, 404, 500, etc.)
http.user_agent         # Filter by user agent
http.host contains "evil"       # Specific host
http.request.uri contains "login" # Specific URI
http.authorization      # HTTP auth headers
http.cookie             # Cookies
http.set_cookie         # Set-Cookie headers

# === TLS/SSL FILTERS ===
tls.handshake.type == 1        # Client Hello
tls.handshake.type == 2        # Server Hello
tls.handshake.extensions_server_name # SNI (server name)
tls.handshake.certificate      # Certificate
tls.record.version == 0x0303    # TLS 1.2

# === SEARCH FILTERS ===
# Search packet content for string (case-sensitive):
frame contains "password"
frame contains "admin"
frame contains "evil.com"

# Case-insensitive search:
tcp.payload contains "User-Agent:"

# === UTILITY FILTERS ===
tcp.stream eq 0                # Specific TCP stream
tcp.analysis.retransmission     # Retransmissions ( congestion or tampering)
tcp.analysis.duplicate_ack      # Duplicate ACKs
tcp.analysis.zero_window        # Zero window (flow control)
tcp.analysis.ack_rtt > 1        # High RTT (latency)

```

10.2 Filter Logic

```

&&    AND    (both conditions must match)
||     OR     (either condition matches)
!()    NOT    (negate condition)
contains (substring match)
matches (regex match)
eq     ==     (equals)
ne     !=     (not equals)
gt     >      (greater than)
lt     <      (less than)
ge     >=     (greater or equal)
le     <=     (less or equal)

```

11. tshark QUICK REFERENCE

11.1 Capture Commands

```
# Basic capture:
tshark -i eth0 -w capture.pcap

# Capture with filter:
tshark -i eth0 -f "port 80 or port 443" -w web.pcap

# Capture specific host:
tshark -i eth0 -f "host 192.168.1.42" -w suspect.pcap

# Rotating capture (100MB files, 50 files max):
tshark -i eth0 -w cap-%Y%m%d-%H%M.pcap -b filesize:100000 -b files:50
```

11.2 Analysis Commands

```
# Read a pcap:
tshark -r capture.pcap

# Read with filter:
tshark -r capture.pcap -Y "http.request"

# Read specific fields:
tshark -r capture.pcap -T fields -e frame.number -e ip.src -e ip.dst -e _ws.col.Protocol -e _ws.col.Info

# Statistics:
tshark -r capture.pcap -q -z io,stat,0          # I/O stats
tshark -r capture.pcap -q -z conv,tcp          # TCP conversations
tshark -r capture.pcap -q -z conv,udp          # UDP conversations
tshark -r capture.pcap -q -z endpoints,ip       # IP endpoints
tshark -r capture.pcap -q -z protocol,fields    # Protocol hierarchy
tshark -r capture.pcap -q -z http,tree         # HTTP stats
tshark -r capture.pcap -q -z dns,tree          # DNS stats

# Follow TCP stream (stream 0):
tshark -r capture.pcap -q -z follow,tcp,ascii,0

# Export objects:
tshark -r capture.pcap --export-objects http,/tmp/http-objects/
tshark -r capture.pcap --export-objects smb,/tmp/smb-objects/
```

11.3 Automation Patterns

```
# Extract all unique IPs:
tshark -r capture.pcap -T fields -e ip.src -e ip.dst | \
  tr '\t' '\n' | sort -u > all_ips.txt

# Extract all DNS queries:
tshark -r capture.pcap -Y "dns.flags.response == 0" \
  -T fields -e dns.qry.name | sort -u > domains.txt

# Extract all HTTP URLs:
tshark -r capture.pcap -Y "http.request" -T fields \
  -e http.host -e http.request.uri | sed 's/\t//' | sort -u > urls.txt

# Extract all User-Agents:
tshark -r capture.pcap -Y "http.user_agent" -T fields \
  -e ip.src -e http.user_agent | sort -u > useragents.txt

# Count packets per protocol:
tshark -r capture.pcap -T fields -e _ws.col.Protocol | \
  sort | uniq -c | sort -rn > protocol_counts.txt
```

```
# Find all SYN scans (SYN without ACK):
tshark -r capture.pcap -Y "tcp.flags.syn == 1 && tcp.flags.ack == 0" \
-T fields -e ip.src -e tcp.dstport | sort > syn_scans.txt
```

12. ATTACK PATTERN RECOGNITION GUIDE

12.1 Quick Pattern Attack Matrix

Pattern	Likely Attack	Filter	Investigation
Many SYN, no data, many ports	Port scan	<code>tcp.flags.syn == 1 && tcp.flags.ack == 0</code>	Source IP = scanner, target ports = services
SYN + RST on a port	Service closed	<code>tcp.flags.reset == 1</code>	Port not available, attacker may try others
SYN + SYN-ACK + data	Service open	<code>tcp.flags.syn == 1 && tcp.flags.ack == 1</code>	Service responded, examine data
TCP 53 with non-DNS payload	Covert shell/tunnel	<code>tcp.port == 53 && !dns</code>	Follow stream — may be shell
Regular small HTTP requests	C2 Beacons	<code>http.request + timing analysis</code>	Calculate interval, identify C2 IP
Large HTTP POST to external	Data exfiltration	<code>http.request.method == POST</code>	Extract POST body, identify data
High-entropy DNS queries	DNS tunneling	<code>udp.port == 53 + inspect query names</code>	Decode query names, extract exfiltrated data
Telnet session	Remote access	<code>telnet</code>	Follow stream — extract commands & credentials

SMB between workstations	Lateral movement	smb2 && ip.src == && ip.dst ==	Check file names, admin shares
RDP between internal hosts	Lateral movement	tcp.port == 3389 && ip.src !=	Unusual RDP sources
Many NXDOMAIN responses	DGA malware	dns.flags.rcode == 3	Source host likely infected with DGA malware
TLS to unknown domains	C2 over TLS	tls.handshake.extensions_server_name	Cross-reference domains with threat intel
ARP broadcast storm	ARP poisoning	arp.opcode == 1 with many requests	Check for MAC spoofing, MITM
ICMP with large payload	Data exfiltration via ICMP	icmp && data.len > 64	Extract data from ICMP payload
FTP credentials in cleartext	FTP credential theft	ftp.request.command == USER \\ \\ ftp.request.command == PASS	Extract credentials, check for unauthorized access

12.2 Common Backdoor Ports

Port	Protocol	Threat Use
53	TCP	DNS tunneling, covert shell
80	TCP	HTTP C2, web shell
443	TCP	Encrypted C2
23	TCP	

Telnet backdoor

4444	TCP	Metasploit default payload (reverse shell)
31337	TCP	Back Orifice backdoor
1234	TCP	Many backdoor Trojans
6667	TCP	IRC botnet C2
8080	TCP	Alternative HTTP C2
8443	TCP	Alternative HTTPS C2
9999	TCP	Various backdoors

12.3 Malware Behavior Signatures in Traffic

Behavior	What You See	Malware Family Examples
DNS beaconing to same domain	Regular DNS queries to one domain	Various DGA families
HTTP GET to /gate.php	C2 check-in panel	Pony, Formgrass, Zeus
POST with encoded data	Data exfiltration	Various info-stealers
TLS to raw IP (no SNI)	C2 avoiding domain IOCs	Cobalt Strike, custom implants
SMB file copy to ADMIN\$	Lateral movement	Emotet, WannaCry, NotPetya
RDP from unexpected source	Lateral movement	Many targeted attacks
DCE/RPC bind requests	Remote execution	PsExec, WMI-based lateral movement
Kerberos AS-REQ bursts	Kerberoasting	Adversary tools



13. LEGAL & CHAIN OF CUSTODY

13.1 Chain of Custody Form Template

EVIDENCE CHAIN OF CUSTODY

Case ID:INC-2024-001

Evidence ID:EVD-001

Description:Network capture (pcap) from core switch SPAN

Source:Core Switch port 24 mirror → sensor eth0

Collected by:[Name], [Role]

Date/Time (UTC):2024-XX-XX HH:MM:SS UTC

Storage Location:/cases/INC-2024-001/original/

File Name:incident-001.pcap

File Size:XX,XXX bytes

SHA-256:[hash value]

MD5:[hash value]

CUSTODY TRANSFER LOG

Date/Time	From	To	Reason
-----	-----	-----	-----

ANALYSIS LOG

Date/Time	Analyst	Action	Result
-----	-----	-----	-----

13.2 Rules of Evidence

- 1. — always work on a copy
- 2. — SHA-256 at minimum, MD5 as secondary
- 3. — what tool, what command, what result
- 4. — use UTC to avoid timezone confusion
- 5. — encrypted, access-controlled, thermally stable
- 6. — every handoff is logged with date/time/person
- 7. — re-hash the copy before starting work
- 8. — hash your analysis output too

13.3 Legal Considerations

Consideration	Action
Privacy laws (GDPR, CCPA)	pcap may contain PII — handle per policy
Authorization	Ensure you have written authorization to capture
Wiretap laws	Some jurisdictions require consent for capture
Internal policy	Follow your organization's IR handling policy
Law enforcement	If criminal case, evidence must meet evidentiary standards
Retention	Follow legal retention requirements for evidence

14. APPENDIX A — PRACTICAL EXERCISES

Exercise 1: Analysis of dns-remoteshell.pcap (This Lab)

Identify a remote shell over multiple ports including DNS port 53.

1. Open dns-remoteshell.pcap in Wireshark
 2. Run Protocol Hierarchy — note Telnet and TCP
 3. Check Conversations — find connections to ports 53, 23, 80, 21
 4. Follow the TCP stream for port 53 — confirm it's a shell, not DNS
 5. Follow the TCP stream for port 23 — extract the banner and commands
 6. Filter for SYN-only packets from 192.168.1.3 — identify the port scan on port 21
 7. Identify the external IPs and their activity
 8. Build a timeline of all events
 9. Extract all IOCs into a table
 10. Write a one-page summary report
- 3 remote shell sessions on ports 53, 23, 80
 - Port scan on 21 (all RST)
 - Windows XP victim
 - Attacker at 192.168.1.3
 - External communications to 83.170.75.178 and 205.227.136.203
 - Commands: dir, ls -la, exit

Exercise 2: DNS Tunneling Detection Challenge

Detect DNS tunneling by analyzing query patterns.

- 1. Download a DNS tunneling pcap (e.g., from Malware Traffic CTF)
- 2. Open in Wireshark
- 3. Filter dns — look at query names
- 4. Identify high-entropy query names (random characters)
- 5. Calculate queries per second — abnormal volume
- 6. Extract all query names — decode base32 encoding
- 7. Reconstruct the exfiltrated data
- 8. Document the C2 domain and extraction technique

Exercise 3: Beaconsing Pattern Detection

Identify C2 beaconsing in network traffic.

- 1. Use a pcap containing beaconsing (e.g., Cobalt Strike beacon traffic)
- 2. Open in Wireshark
- 3. Filter for outbound HTTP/HTTPS to external IPs
- 4. Sort by time — observe regular intervals
- 5. Use Statistics I/O Graph — visualize the periodic pattern
- 6. Calculate the beacon interval (e.g., 30s, 60s)
- 7. Extract the C2 domain/IP
- 8. Map to MITRE ATT&CK: Command and Control Beaconsing

15. APPENDIX B — TOOL ARSENAL

15.1 Wireshark vs tshark Decision Guide

Use Case	Tool
Interactive exploration	Wireshark
Following TCP streams	Wireshark
Protocol hierarchy overview	Wireshark (GUI is faster)
Exporting objects (HTTP, SMB)	Wireshark
Flow graph visualization	Wireshark

I/O Graph visualization	Wireshark
Automation / scripting	tshark
Bulk IOC extraction	tshark
Processing multiple pcaps	tshark
CI/CD pipeline integration	tshark
Remote/headless analysis	tshark
Quick field extraction	tshark (-T fields)
Custom display filters	Both (same syntax)

15.2 Complementary Tools

Tool	Purpose	When to Use
NetworkMiner	Auto-extract files, images, credentials	Quick triage of many pcaps
zeek (Bro)	Protocol-level logging at scale	Enterprise network monitoring
Suricata/Snort	IDS/IPS signature matching	Real-time alerting on known threats
binwalk	Firmware/embedded file analysis	Extract files from payloads
foremost	File carving from pcaps	Recover deleted/hidden files
volatility	Memory forensics correlation	Cross-reference RAM artifacts with network
CyberChef	Decode/encode (base64, hex, XOR)	Decoding payloads and data
MITRE ATT&CK Navigator	Visualize attack mapping	Reporting and presentation
VirusTotal	Check file hashes, IPs, domains	Threat intelligence enrichment
AbuseIPDB	Check IP reputation	

External IP assessment

urlscan.io	Scan suspicious URLs	Analyze web infrastructure
Security Onion	All-in-one NSM platform	Enterprise-scale detection & response
Arkime	Full packet capture + indexing	Large-scale pcap search and analysis
RITA	Beaconing detection	Analyze Zeek logs for C2 patterns
stenoographer	Full packet capture storage	Large volume pcap storage and retrieval

15.3 Recommended Learning Path

Beginner:

- └─ Wireshark basics (filters, following streams)
- └─ Understanding TCP/IP, DNS, HTTP, TLS
- └─ Analyze simple pcaps (malware samples)

Intermediate:

- └─ tshark automation and scripting
- └─ Protocol anomaly detection
- └─ DNS tunneling analysis
- └─ Beaconing detection
- └─ Attack pattern recognition (MITRE ATT&CK)

Advanced:

- └─ Zeek log analysis at scale
- └─ Suricata rule writing
- └─ Memory + network correlation
- └─ Timeline reconstruction across multiple sources
- └─ Full-scale enterprise incident response
- └─ Threat hunting with network telemetry

15.4 Practice Resources

Resource	URL	Type
Malware Traffic CTF	malware-traffic-analysis.net	Real-world malware pcaps
PCAP Challenges	pcap1844.blogspot.com	Practice captures
Netresec Samples	netresec.com/?page=PCAPfiles	Sorted by malware type
Wireshark Sample Captures	wiki.wireshark.org/Downloads	All types

Stratosphere IPS	stratosphereips.org	MalMalware datasets
CIC IDS 2017	unb.ca/cic/datasets/ids.html	Labeled intrusion detection dataset
Zeek Sample Logs	github.com/corelight	Zeek log examples

FINAL CHECKLIST: BEFORE YOU SUBMIT YOUR ANALYSIS

- — Hash of working copy matches original
- — Protocol hierarchy reviewed
- — Conversation table reviewed
- — Cross-referenced with threat intel
- — Checked for tunneling/DGA
- — Checked for C2, exfiltration
- — Commands extracted
- — If present, documented
- — Exported and hashed
- — Events ordered, attack chain mapped
- — Full table with context
- — Tactics and techniques identified
- — Executive summary + technical details
- — Every action logged
- — Original in secure storage, working copy encrypted

This playbook is a living document. Update it with new techniques, tools, and lessons learned from each engagement.

End of Playbook — Version 1.0